

Project Title

Surface Crack Detection Using Convolutional Neural Networks with Transfer Learning

Submitted By: Avishek Bhowmik

M.Sc. in Software Engineering

University of Europe for Applied Sciences

Supervisor: Prof. Raja Hashim Ali

Date: 07-06-2026

Project Links:

GitHub Repository: <https://github.com/avishekbhowmik/surface-crack-detection.git>

Kaggle: <https://www.kaggle.com/code/avishekbhowmik/surface-crack-detection-ss26-pr>

1. Background and Motivation

Concrete infrastructure like bridges, roads, tunnels, and buildings undergoes progressive mechanical degradation over time. Surface cracks are among the earliest and most visually identifiable signs of structural damage. When left undetected, micro-cracks can propagate into larger fractures that pose serious safety risks and require expensive repairs.

Traditional inspection methods rely on manual visual surveys conducted by engineers. This process is time-consuming, costly, and highly subjective. Automated inspection using deep learning, specifically Convolutional Neural Networks (CNNs), offers a reliable and scalable alternative. CNNs are well-suited to this task because they are capable of learning spatial features directly from image data through hierarchical convolution and pooling operations.

This project applies CNN-based image classification to automate crack detection. Two approaches are explored: a custom CNN trained from scratch as a baseline and MobileNetV2 using transfer learning. Explainability methods are also incorporated to verify that the model's predictions are based on meaningful visual evidence, rather than irrelevant image regions.

2. Problem Statement

This project addresses the binary classification of concrete surface images into two categories:

- Positive - the image contains a visible surface crack
- Negative - the image shows an intact, crack-free surface

The model must satisfy the following requirements:

- Achieve strong but realistic validation accuracy, kept deliberately below 90% through regularisation and dropout to avoid overfitting
- Compare a custom-built CNN against a transfer learning model to evaluate the benefit of pre-trained weights
- Apply Grad-CAM and SHAP to verify that predictions rely on the crack regions in the image
- Accept any new uploaded image and return the predicted class along with a crack probability score

3. Selected Dataset and Dataset Link

Dataset Name: Surface Crack Detection

Source: Kaggle

Link: <https://www.kaggle.com/datasets/arunrk7/surface-crack-detection>

4. Dataset Description

Attribute	Details
Total Images	40,000
Number of Classes	2 (Binary Classification)
Class Names	Positive (crack present) and Negative (no crack)
Images per Class	20,000 Positive and 20,000 Negative
Class Distribution	Perfectly balanced - 50% per class, no imbalance
Native Resolution	227 × 227 pixels

Resized to (training)	96 × 96 pixels for faster processing
Image Type	RGB colour JPEG photographs
Train / Val Split	80% training (32,000) and 20% validation (8,000)
Source	Real-world photographs of concrete surface specimens

5. Research Questions

This project investigates the following research questions:

RQ1: How accurately can a custom CNN classify concrete surface cracks while keeping validation accuracy below 90%?

RQ2: Does transfer learning using MobileNetV2 (feature extraction mode) improve classification performance compared to the custom CNN?

RQ3: Which model provides the best trade-off between classification accuracy and computational cost in terms of training time and number of parameters?

RQ4: Which image regions does each model focus on when making correct predictions, as revealed by Grad-CAM heatmaps?

RQ5: Do Grad-CAM attention patterns differ between the custom CNN and MobileNetV2, and are they focused on actual crack regions?

RQ6: What positive and negative visual evidence do SHAP explanations reveal about individual model decisions?

RQ7: Can explainability methods help identify and understand why specific images are misclassified?

6. Expected Results

Based on the dataset characteristics and the nature of binary image classification, the following performance outcomes are anticipated. Validation accuracy is intentionally limited below 90% through regularisation techniques to prevent overfitting.

Model	Val Accuracy	AUC-ROC	Train Time	Parameters
Custom CNN (baseline)	82-87%	0.88–0.93	Fastest	~93K
MobileNetV2 (Transfer)	85-89%	0.91–0.96	Moderate	~155K trainable

MobileNetV2 is expected to outperform the custom CNN because it begins from ImageNet weights, providing a much better starting point than random initialisation. The custom CNN, while less accurate, is expected to converge faster due to its smaller number of parameters.

7. Proposed Methodology

7.1 Overall Pipeline

The project follows a supervised deep learning pipeline designed for binary image classification. The methodology is divided into eight stages, each described in the subsections below.

1. Dataset loading with automatic path detection for Positive and Negative class folders

2. Preprocessing: resize images to 96×96 pixels, normalise pixel values to [0.0, 1.0]
3. Build a tf.data pipeline with parallel decoding, training-only augmentation, and prefetching
4. Design and train a custom CNN baseline model from scratch
5. Apply MobileNetV2 transfer learning in feature extraction mode with frozen base layers
6. Evaluate both models using accuracy, precision, recall, F1, AUC-ROC, confusion matrix, and training time
7. Apply Grad-CAM to visualise attention regions for correct and misclassified predictions
8. Apply SHAP to quantify positive and negative regional contributions to individual predictions
9. Implement new image inference: accept any uploaded image and return class and crack probability
10. Save all figures as high-resolution PDF files, comparison table as CSV, and best model as .keras

7.2 Dataset Loading and Path Detection

The dataset is loaded from the Kaggle input directory using an automatic path detection function that recursively walks the directory tree to locate the folder containing the Positive and Negative subfolders. All image file paths are collected with their corresponding integer labels, where 0 represents Negative (no crack) and 1 represents Positive (crack present). This approach ensures the code runs without manual path configuration regardless of how the dataset is attached to the Kaggle notebook.

7.3 Data Preprocessing and Pipeline

Images are preprocessed using a tf.data pipeline. Each image is decoded from JPEG format, resized from its native resolution of 227×227 pixels to 96×96 pixels, and normalised by dividing all pixel values by 255.0, producing a float32 tensor in the range [0.0, 1.0]. The dataset is split into 80% training (32,000 images) and 20% validation (8,000 images) using a fixed random seed of 42 for reproducibility. The pipeline uses AUTOTUNE parallel mapping and prefetching to overlap CPU decoding with GPU training, eliminating idle time between batches.

Step	Implementation
Image resizing	tf.image.resize to 96×96 during parallel decoding
Normalisation	Pixel values divided by 255.0, output range [0.0, 1.0]
Train/Val split	80/20 random split, seed = 42 for reproducibility
Augmentation (train)	Random horizontal flip, vertical flip, brightness (± 0.1), contrast (0.9-1.1)
Augmentation (val)	None - validation images are only rescaled to prevent data leakage
Batch size	128 images per batch for efficient GPU utilisation
Pipeline optimisation	tf.data with AUTOTUNE parallel map and prefetch

7.4 Custom CNN Design and Training

A three-block convolutional neural network is designed from scratch as the baseline model. Each block consists of a Conv2D layer with ReLU activation followed by MaxPooling2D. The filter counts increase progressively from 32 to 64 to 128 across the three blocks to capture increasingly complex spatial features. L2 regularisation with $\lambda = 0.01$ is applied to all convolutional layers. The classification head uses GlobalAveragePooling2D instead of Flatten, which reduces the parameter count and further limits overfitting. A Dropout layer with rate 0.5 precedes

the final Dense output neuron with Sigmoid activation, which produces the crack probability score in the range [0.0, 1.0]. The model is compiled with the Adam optimiser and binary cross-entropy loss. Training runs for a maximum of 10 epochs with EarlyStopping (patience = 4) restoring the best weights, and ReduceLROnPlateau (factor = 0.3, patience = 2) reducing the learning rate when validation loss plateaus.

7.5 Transfer Learning with MobileNetV2

MobileNetV2, pre-trained on ImageNet, is used as the second model in feature extraction mode. The base model is loaded with `include_top=False` to remove the original 1000-class classification head, and all base layers are frozen by setting `trainable=False`. A new classification head is appended consisting of GlobalAveragePooling2D, a Dense layer with 128 neurons and ReLU activation with L2 regularisation ($\lambda = 0.01$), followed by Dropout with rate 0.7, and a final Dense output neuron with Sigmoid activation. Only the head parameters (approximately 155,649) are updated during training while the 2,257,984 base parameters remain fixed. The same training configuration as the custom CNN is used, with fresh callback instances created for each model to prevent state carry-over between training runs.

7.6 Model Evaluation

Both models are evaluated on the held-out validation set using accuracy, precision, recall, F1 score, AUC-ROC, confusion matrix, training time, and number of trainable parameters. The confusion matrix is visualised as a heatmap using Seaborn. ROC curves for both models are plotted on the same axes with AUC values displayed in the legend. A model comparison table is saved as a CSV file containing validation accuracy, AUC, training time, and parameter count for both models.

7.7 Explainability

Two explainability methods are applied to interpret model predictions. Grad-CAM is implemented using a manual layer-by-layer forward pass with GradientTape to compute input gradients, which are averaged across colour channels to produce a spatial heatmap. The heatmap is overlaid on the original image using the jet colourmap to highlight which regions most influenced the prediction. Grad-CAM is applied to both correctly classified and misclassified images. SHAP is applied using GradientExplainer with a background set of 50 randomly selected validation images. SHAP values are computed for three test samples and visualised as colour-coded overlays where red regions contribute positively toward a crack prediction and blue regions contribute negatively. All explainability figures are saved as high-resolution PDF files (dpi = 300).

7.8 New Image Inference

A dedicated inference function accepts any new image path, loads and preprocesses the image to match the model input format (96×96 pixels, normalised), runs a forward pass through the best model, and returns the predicted class along with the crack probability percentage. The image is displayed with the prediction label and probability overlaid as the title. This function forms the core of the Phase 3 web application, where users upload images through a browser interface and receive real-time predictions.

8. CNN Model Design

8.1 Activation Functions

Location	Activation	Reason
Input layer	None	Input layer only passes data forward as no activation is needed
After each Conv2D	ReLU	Removes negative convolution outputs; standard for all convolutional layers
Hidden dense layers	ReLU	Unbounded; preserves large feature values without suppression
Output layer	Sigmoid	Maps output to [0, 1] probability for binary classification
MobileNetV2 base	ReLU6	Built into the pre-trained base; capped at 6 for efficiency

8.2 Custom CNN Architecture

A three-block feature extractor is designed from scratch, followed by a GlobalAveragePooling2D layer and a dropout-regularised head output. L2 regularisation is applied to all Conv2D layers to keep validation accuracy below 90%.

SL.	Layer	Output Shape	Parameters	Notes
1	Input	$96 \times 96 \times 3$	-	RGB image
2	Conv2D (32 filters, 3×3) + ReLU	$94 \times 94 \times 32$	896	L2 $\lambda = 0.01$
3	MaxPooling2D (2×2)	$47 \times 47 \times 32$	-	Downsampling
4	Conv2D (64 filters, 3×3) + ReLU	$45 \times 45 \times 64$	18,496	L2 $\lambda = 0.01$
5	MaxPooling2D (2×2)	$22 \times 22 \times 64$	-	Downsampling
6	Conv2D (128 filters, 3×3) + ReLU	$20 \times 20 \times 128$	73,856	L2 $\lambda = 0.01$
7	MaxPooling2D (2×2)	$10 \times 10 \times 128$	-	Downsampling
8	GlobalAveragePooling2D	128	-	Replaces Flatten
9	Dropout (0.5)	128	-	Prevents overfitting
10	Dense (1) + Sigmoid	1	129	Crack probability

Total trainable parameters: approximately 93,377

Optimiser: Adam | **Loss:** Binary Cross-Entropy | **Max epochs:** 10 | **Early Stopping patience:** 4

9. Transfer Learning Models

9.1 Motivation

Training a deep CNN from scratch requires large amounts of labelled data to avoid overfitting. When data is limited or the training time needs to be reduced, transfer learning offers an effective solution. A model pre-trained on a large general-purpose dataset already contains learned feature representations for shapes, edges, and textures. These low-level features are universal and directly applicable to crack detection in concrete images.

In this project, MobileNetV2 is used for feature extraction, with the pre-trained base layers frozen and only the added classification head trained. This approach is faster and less prone to overfitting than fine-tuning.

9.2 Feature Extraction vs. Fine-Tuning

Property	Feature Extraction (used)	Fine-Tuning (not used)
Base layers	Frozen - weights do not change	Some or all layers unfrozen
What is trained	Classification head only	Head and selected base layers
Training speed	Faster (fewer trainable parameters)	Slower (more parameters)
Risk	Lower risk of overfitting	Higher risk with small datasets
Accuracy	Sufficient for this dataset	Potentially higher but not needed

9.3 MobileNetV2 Configuration

Property	Value
Pre-trained on	ImageNet (1.2 million images, 1000 classes)
include_top	False - original 1000-class output layer removed
Base layers	Frozen (trainable = False)
Added head	GlobalAveragePooling2D → Dense(128, ReLU, L2=0.01) → Dropout(0.7) → Dense(1, Sigmoid)
Input shape	96 × 96 × 3
Trainable parameters	Approximately 155,649 (head only)
Frozen parameters	Approximately 2,257,984 (base)

10. Evaluation Metrics

Metric	Definition	Purpose
Accuracy	Correct predictions / Total predictions	Overall classification performance
Precision	TP / (TP + FP)	Measures false alarm rate
Recall	TP / (TP + FN)	Measures how many cracks are detected
F1 Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{P} + \text{R})$	Balanced measure of precision and recall
AUC-ROC	Area under the ROC curve	Threshold-independent separability
Confusion Matrix	2×2 table: TP, TN, FP, FN	Visual breakdown of classification errors
Training Time	Wall-clock seconds from training start to finish	Computational cost comparison
Trainable Params	Number of weight values updated during training	Model complexity measure
Grad-CAM Quality	Qualitative: High / Medium / Low (focus on cracks)	Explainability trust assessment
SHAP Clarity	Qualitative: degree of region-level explanation	Positive and negative contribution clarity

11. Explainability Methods

11.1 Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) generates a heatmap that highlights which regions of the input image most influenced the model's prediction. It works by computing the gradients of the predicted class score with respect to the output of the last convolutional layer. Regions shown in red and yellow receive the highest attention; blue regions are less relevant.

Grad-CAM is applied to both correctly classified and misclassified images. For a trustworthy model, the heatmap should consistently focus on the actual crack region rather than on the image background or irrelevant areas.

11.2 SHAP

SHAP (SHapley Additive exPlanations) divides the image into regions and quantifies how much each region contributes, positively or negatively, to the final prediction. Red regions push the prediction toward the crack class; blue regions push it away. SHAP provides a more detailed explanation than Grad-CAM but is computationally slower.

GradientExplainer is used with a background set of 50 validation images.

Property	Grad-CAM	SHAP
What it shows	Which regions the model attends to	How much each region contributes
Direction	Attention level only (no sign)	Positive and negative contributions
Speed	Fast	Slower (requires background set)
Works on	CNN image models only	Any model type including tabular data
Output	Heatmap overlay on original image	Colour-coded contribution overlay

12. Limitations and Future Work

Current Limitations

- The model performs binary classification only and does not estimate crack severity or width
- The dataset is limited to concrete surfaces; performance on asphalt, metal, or glass is untested
- Images are of uniform resolution; real-world deployment may involve varying aspect ratios and lighting conditions
- SHAP computation is slow on large validation sets due to the background reference requirement

Future Work

- Extend to multi-class classification: hairline, moderate, and severe crack categories
- Apply fine-tuning to the top layers of MobileNetV2 for potential accuracy improvements
- Add VGG16 and ResNet50 for a broader transfer learning comparison
- Integrate the web application with a mobile camera API for real-time on-site inspection
- Explore model quantisation and pruning for deployment on low-resource edge devices

13. References

1. Arunrk7 (2019). Surface Crack Detection Dataset. Kaggle.
2. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.
3. Howard, A. G., et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv:1704.04861.
4. Sandler, M., et al. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018.
5. Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV 2017.
6. Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS 2017.
7. Zhang, L., et al. (2016). Road Crack Detection Using Deep Convolutional Neural Network. ICIP 2016.
8. Chollet, F. (2021). Deep Learning with Python, 2nd Edition. Manning Publications.
9. Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv:1409.1556.